hashlock.



Security Audit

Vana V (Decentralized Data Economy)

December 2025 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	13
Centralisation	21
Conclusion	22
Our Methodology	23
Disclaimers	25
About Hashlock	26

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Vana team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

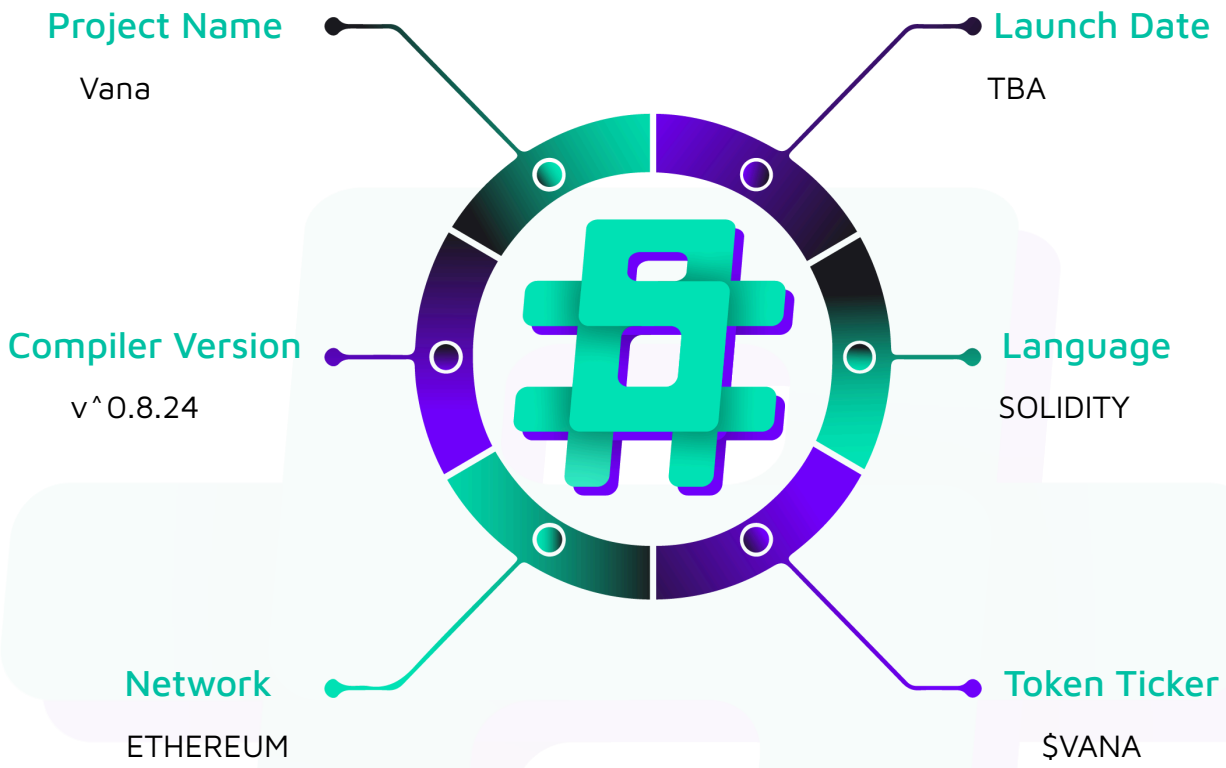
Project Context

Vana is positioning itself as a next-generation data economy platform, focused on user-owned data. It offers tools and services where individuals can contribute their data, launch data collectives, tokenize data assets, and trade data tokens. The core vision is to shift from "data extraction" (where platforms harvest and own your data) to a model where data is owned and controlled by its creators/users, thus enabling participation, value sharing, and transparency.

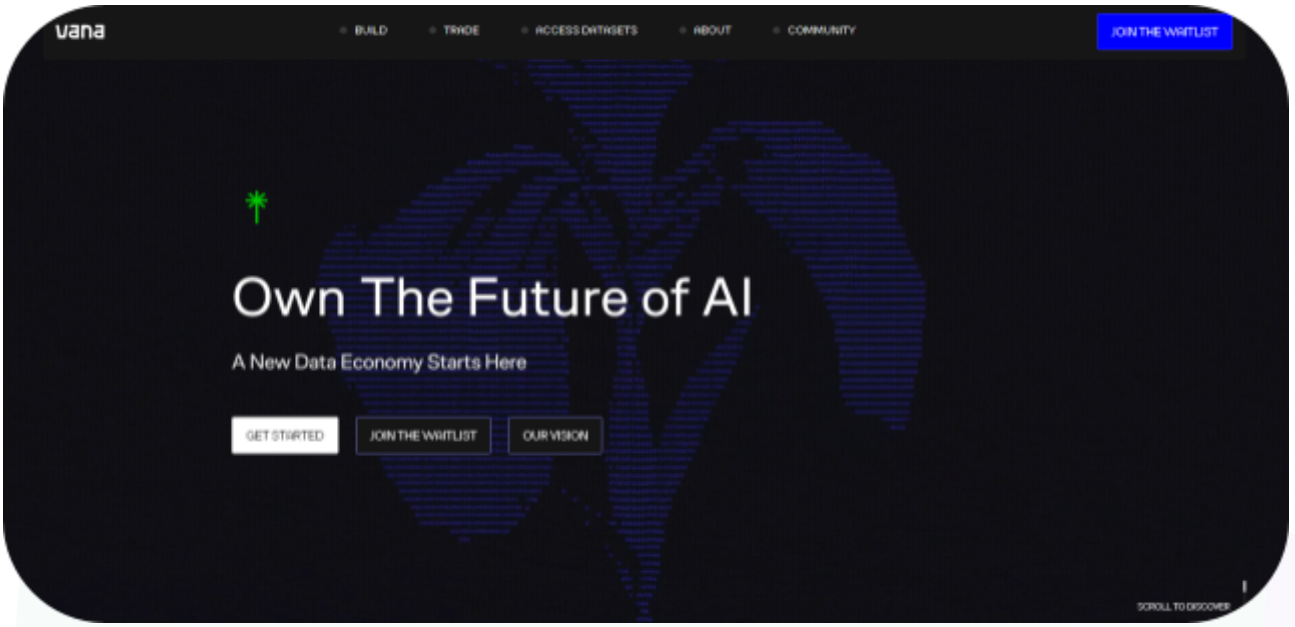
Project Name: Vana
Project Type: Decentralized Data Economy .
Compiler Version: ^0.8.24
Website: https://www.vana.org/
Logo:

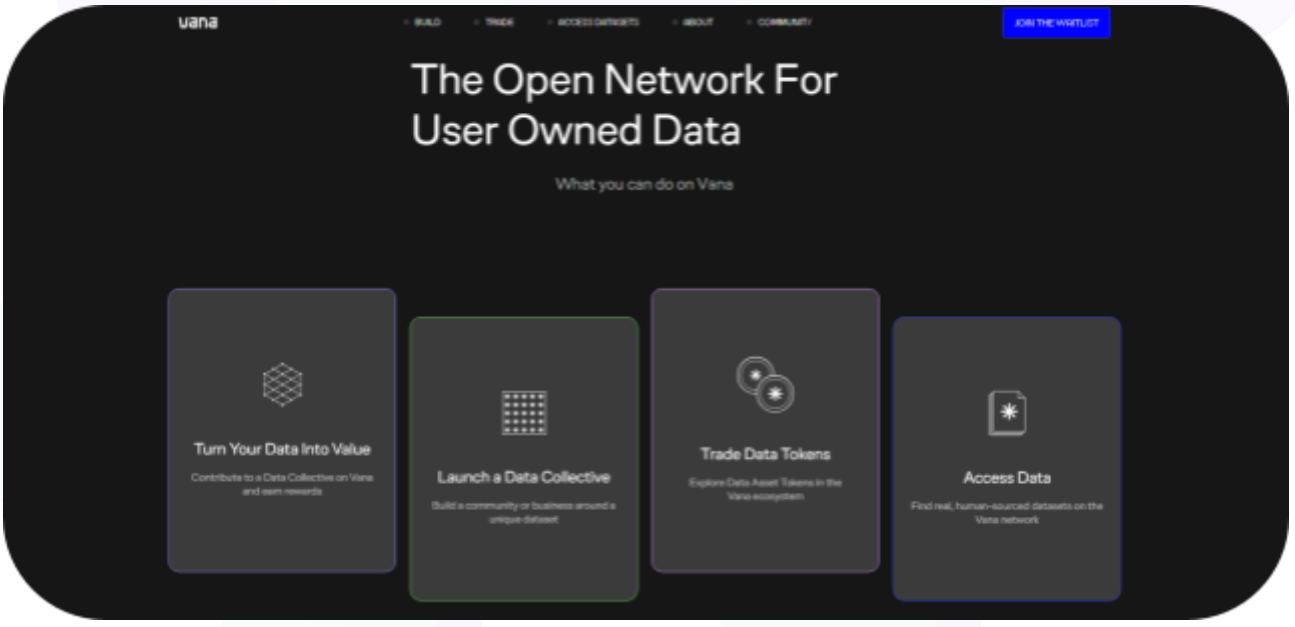


Visualised Context:



Project Visuals:





Audit Scope

We at Hashlock audited the solidity code within the Vana project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**Vana Smart Contracts**
Platform	**Ethereum / Solidity**
Audit Date	**December, 2025**
Scope	VanaPoolEntity contracts/vanaStaking/vanaPoolEntity/VanaPoolEntityImplementation.sol
Contract 1	VanaPoolStakingImplementation.sol
Contract 2	VanaPoolEntityImplementation.sol
Contract 3	VanaPoolStakingStorageV2.sol
Contract 4	IVanaPoolStaking.sol
Audited GitHub Commit Hash	fe8b1ed0fa05a7ab03983ed627c15a5a97a5bfd3
Fix Review GitHub Commit Hash	2327e0aa4c3c32d164fe828941f23b01d73b4dfe

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

4 Low severity vulnerabilities

2 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
VanaPoolStakingImplementation.sol - Allows user to: - Stake VANA with bonding period tracking - Unstake by share amount or VANA amount - View max unstakeable amount - View accruing interest and earned rewards - Forfeit rewards if unstaking during bonding - Allows maintainer to: - Update the bonding period	**Contract achieves this functionality.**
VanaPoolEntityImplementation.sol - Allows VanaPoolStaking to: - Return forfeited rewards to locked pool	**Contract achieves this functionality.**

Code Quality

This audit scope involves the smart contracts of the Vana project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Vana project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Low

[L-01] VanaPoolStakingImplementation#stake - Dust stake causes division-by-zero for new stakers

Description

When a new staker (with 0 existing shares) stakes a dust amount into an entity with a high share price, the transaction reverts due to division by zero.

Vulnerability Details

The `stake()` function calculates `sharesIssued` based on the current share price. When `sharesIssued` rounds to 0 for a new staker, `newTotalShares` becomes 0, leading to `newTotalValue = 0` and subsequent division by zero.

```
uint256 sharesIssued = (vanaToShare * stakeAmount) / 1e18;  // Can be 0

// ...

uint256 newTotalShares = stakerEntity.shares + sharesIssued;  // 0 + 0 = 0 for new staker

uint256 newTotalValue = (newTotalShares * shareToVana) / 1e18;  // = 0

// ...

stakerEntity.rewardEligibilityTimestamp = currentTimestamp + (stakeAmount * bondingPeriod) / newTotalValue;  // DIV BY ZERO
```

File: `contracts/vanaStaking/vanaPoolStaking/VanaPoolStakingImplementation.sol:489`

Impact

Transaction reverts and user loses gas fees. No fund loss occurs. This is a self-DOS on an edge case and is not exploitable. Slippage protection (`shareAmountMin > 0`) prevents this if used.

Recommendation

We recommend adding a check for zero shares issued in the `stake()` function.

```
if (sharesIssued == 0) {

    revert InsufficientStakeAmount();

}
```

Status

Resolved

[L-02] VanaPoolStakingImplementation#getAccruingInterest - View functions return incorrect values for entity owners

Description

Entity owners registered via `registerEntityStake()` have `costBasis = 0`, causing the new view functions to incorrectly report their entire position as "interest" or "rewards".

Vulnerability Details

The `registerEntityStake()` function only sets `shares` but not `costBasis`. This causes the new `getAccruingInterest()` and `getEarnedRewards()` functions to calculate incorrect values.

```
// In getAccruingInterest and getEarnedRewards:

if (currentValue > stakerEntity.costBasis) {

     pendingInterest = currentValue - stakerEntity.costBasis;   // = currentValue - 0 = ENTIRE position

}
```

File: `contracts/vanaStaking/vanaPoolStaking/VanaPoolStakingImplementation.sol:314-355`

Note: The bonding period bypass for entity owners is likely intentional design since entity creation requires `MAINTAINER_ROLE` approval and entity owners are trusted Data Validators. However, `costBasis` should still be set for correct view function behavior.

Impact

Frontend displays incorrect reward amounts for entity owners. No fund loss occurs as actual unstake logic works correctly.

Recommendation

We recommend setting `costBasis` in the `registerEntityStake()` function.

```
// In getAccruingInterest and getEarnedRewards:

if (currentValue > stakerEntity.costBasis) {

     pendingInterest = currentValue - stakerEntity.costBasis;   // = currentValue - 0 = ENTIRE position

}
```

Status

Resolved

[L-03] VanaPoolStakingImplementation#updateBondingPeriod - Missing event emission

Description

The new `updateBondingPeriod()` admin function does not emit an event, making it difficult to track parameter changes off-chain.

Vulnerability Details

```
// VanaPoolStakingImplementation.sol:386-388

function updateBondingPeriod(uint256 newBondingPeriod) external onlyRole(MAINTAINER_ROLE)
{ bondingPeriod = newBondingPeriod;

    // No event emitted}
```

File:`contracts/vanaStaking/vanaPoolStaking/VanaPoolStakingImplementation.sol:386-388`

Impact

Off-chain monitoring and audit trails are incomplete. Bonding period changes cannot be easily tracked or verified.

Recommendation

We recommend adding an event for the bonding period update.

```
event BondingPeriodUpdated(uint256 oldBondingPeriod, uint256 newBondingPeriod);

function updateBondingPeriod(uint256 newBondingPeriod) external onlyRole(MAINTAINER_ROLE)
{   uint256 oldBondingPeriod = bondingPeriod;

    bondingPeriod = newBondingPeriod;

    emit BondingPeriodUpdated(oldBondingPeriod, newBondingPeriod);}
```

Status

Resolved

[L-04] VanaPoolStakingImplementation#updateBondingPeriod - No maximum limit on bonding period

Description

The new `updateBondingPeriod()` function has no upper bound validation.

Vulnerability Details

The `same` here

```
function updateBondingPeriod(uint256 newBondingPeriod) external onlyRole(MAINTAINER_ROLE)
{

    bondingPeriod = newBondingPeriod;

}
```

Admin could set to `type(uint256).max`, causing overflow in bonding calculations or locking all new stakes forever.

File: `contracts/vanaStaking/vanaPoolStaking/VanaPoolStakingImplementation.sol:386-388`

Impact

Relies on admin trust. Accidental misconfiguration could lock stakes.

Recommendation

We recommend adding a reasonable upper bound (e.g., `MAX_BONDING_PERIOD = 365 days`).

Status

Resolved

QA

[Q-01] VanaPoolStakingImplementation#_unstake - TODO comment left in production code

Description

A `TODO` comment in the new `_unstake()` function indicates incomplete functionality.

Vulnerability Details

```
// VanaPoolStakingImplementation.sol:611-623

//todo: block owner from unstaking below registration stake

//        uint256 vanaToShare = vanaPoolEntity.vanaToEntityShare(entityId);

//        ...

//        if (entityInfo.ownerAddress == staker) {

//            if (stakerShares - shareAmount < ownerMinShares) {

//                revert CannotRemoveRegistrationStake();

//            }

//        }
```

File: `contracts/vanaStaking/vanaPoolStaking/VanaPoolStakingImplementation.sol:611-623`

Impact

Indicates planned entity owner protection not yet implemented. Entity owners can currently unstake below registration stake.

Recommendation

We recommend completing or removing `TODO` items before production deployment.

Status

Acknowledged

[Q-02] VanaPoolStakingImplementation#unstakeVana - Double processRewards call

Description

The new `unstakeVana()` function calls `processRewards()`, then calls `_unstake()` which calls it again.

Vulnerability Details

```
function unstakeVana(...) external nonReentrant whenNotPaused {

    vanaPoolEntity.processRewards(entityId);  // First call (line 553)

    _unstake(staker, entityId, shareAmount, 0);  // Calls processRewards AGAIN (line 609)

}
```

The code comment at line `585` acknowledges this: "`processRewards` returns early since already processed"

File: `contracts/vanaStaking/vanaPoolStaking/VanaPoolStakingImplementation.sol:553,609`

Impact

Wastes approximately 2,600 gas per call from external call overhead. Second call returns early at `timeElapsed == 0` check but still incurs call overhead.

Recommendation

We recommend adding a flag parameter to skip processRewards in `_unstake()` when already called, or refactoring to avoid the double call.

Status

Resolved

Centralisation

The Vana project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Conclusion

After Hashlock's analysis, the Vana project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au

